Full-Duplex Voice Agent on a Raspberry Pi

sections 1-6 in priority order

1 SYSTEM DIAGRAM *the one diagram that explains your build*

PASTE YOUR SYSTEM DIAGRAM HERE

audio path - MCP servers - harness - cloud APIs - barge-in

```
voice in -> VAD -> ASR -> LLM(+tools) -> TTS -> voice out
```

barge-in: speech detected -> stop TTS + LLM -> listen

- Mic -> VAD (on RPi) -> ASR MCP -> agent -> TTS MCP -> speaker
- Harness with MCP client: onit / pipecat / crewAI
- LLM may be a commercial API; tools via YOUR MCP servers
- All MCP servers built by you and running on the RPi
- Barge-in cancels TTS playback and the LLM stream
- Show the on-RPi vs cloud boundary clearly

2 PROMPT *keep it in the repo, version it*

SYSTEM PROMPT (v#) - excerpt

```
You are <name>, a <persona> voice agent on a RPi.
Short, natural sentences; max 2 per turn.
If the user speaks, stop and listen immediately.
Confirm before costly or irreversible actions.
Use MCP tools: <asr, tts, task tools>.
Never read markdown or URLs aloud.
```

3 CONTEXT ENGINEERING

GOES IN:

- system prompt + persona
- ASR partials/finals (rolling)
- VAD turn events
- MCP tool results
- memory / RAG (opt.)

MANAGED:

- token budget; trim/summarize
- cache stable prefix (TTFT)
- idle->listen->think->speak
- on barge-in: drop stale audio
- log context size per run

MCP Services, Evaluation & Model Serving

build + run servers on the RPi

4 MCP SERVICES *+ section 6: model serving (one table, one truth)*

Component	MCP server / transport	Tools	Model (example)	Served where / notes
ASR	asr-mcp / stdio, HTTP	transcribe(audio) -> text	*<SOTA ASR API>*	RPi server; streaming partials, final on VAD end
TTS	tts-mcp / stdio, HTTP	speak(text) -> audio	*<SOTA TTS API>*	RPi server; stream chunks, cancel on barge-in
LLM	(harness MCP client)	tools / function-calling	*<GPT-4o / Claude ...>*	commercial API; streaming tokens
VAD	in-process	detect_speech(frames)	Silero VAD	RPi on-device, 16 kHz mono
Task	*<your-tool-mcp>*	*<your schema>*	*<n/a / API>*	RPi server; add one row per tool

5 EVALUATION & BENCHMARK *standardized latency method (classwork)*

- One clock: RPi monotonic; timestamp every stage
- VAD end -> ASR final -> LLM 1st token -> TTS 1st audio -> speaker
- E2E = speaker - VAD end; barge-in = stop - speech start
- *<N> phrases x <M> runs; report mean + p95; JSON log per run*

Metric	Definition	Target	Mean	p95
E2E latency	end of speech -> audio out			
ASR final	end of speech -> transcript			
LLM TTFT	transcript -> first token			
TTS TTFB	text -> first audio chunk			
Barge-in	speech -> playback stopped			

on

DEMO (deck 2)

script 2-3 min: real task,
natural turns (no push-to-talk),
barge-in, live MCP tool call,
recovery when an API is slow

RESULTS (deck 2)

fill Mean/p95 in table above;
paste latency chart
state: RPi model/RAM, cooling,
network, date, model versions

CHALLENGES & REFERENCES

1 challenge per row, each with
a fix + the measurement that
proves it worked
refs: modelcontextprotocol.io,
harness docs, logs, repo link